



中序 inorder LVR
前序 preorder VLR
後序 postorder LRV

1. (10%) If given three nodes A, B, C:

- (1) How many different binary trees can be constructed? $\frac{1}{3+1} C_3^6 = \frac{1}{4} \cdot \frac{6!}{3!3!} = 5$
- (2) How many different ordered trees can be constructed? 5
- (3) How many different unordered trees (also known as oriented trees) can be constructed? 5
- (4) How many different free trees (i.e., connected acyclic graphs) can be constructed? $3! = 6$
- (5) If one of the pre-order traversal, in-order traversal, or post-order traversal of the three nodes is ABC, how many different binary trees can be constructed? 1

2. (a) (5%) Please represent the following tree as a binary tree.

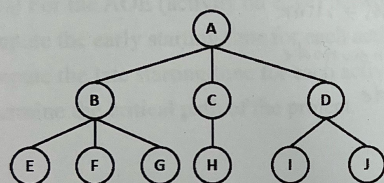


Fig. (a)

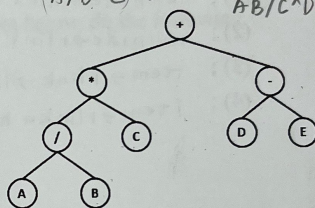


Fig. (b)

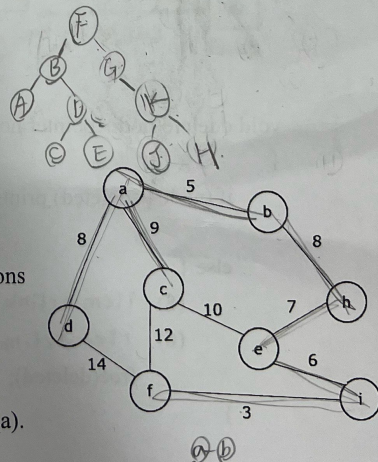
- (c) (5%) Based on the results of in-order traversal and post-order traversal, draw the binary tree.

In-order: ABCDEFGHJK
Post-order: ACEDBJHKGF

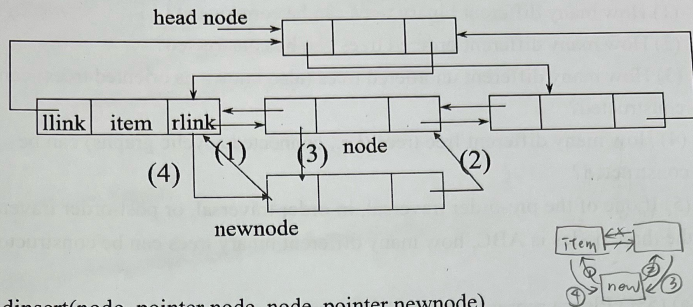
3. For the given graph as shown on the right.

- (a) (15%) Please use the Kruskal's, Prim's, and Sollin's algorithms to find the minimum cost spanning tree. Please provide detailed explanations for step by step.

- (b) (10%) Write pseudo-code to describe how to obtain the result of the Kruskal's algorithm for (a).

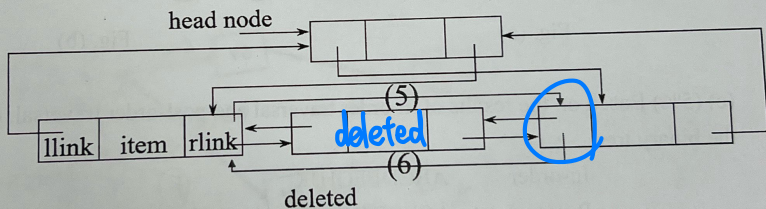


4. (24%) Complete the functions for inserting and deleting nodes in a doubly circular linked list, as shown in the diagram below.



```
void dinsert(node_pointer node, node_pointer newnode)
```

```
{
    (1); newnode → llink = item
    (2); newnode → rlink = item → rlink
    (3); item → rlink → llink = newnode
    (4); item → rlink = newnode
}
```



```
void ddelete(node_pointer node, node_pointer deleted)
```

```
{
    if (node == deleted) printf("Deletion of head node
                             not permitted.\n");
    else {
        (5) item → rlink = item → rlink → rlink
        (6) item → rlink → llink = item
        free(deleted);
    }
}
```



```

    if (rootPtr == NULL) return;
    else {
        if (rootPtr->key <= b && rootPtr->key >= a) v. push_back(rootPtr->key);
        range_count(rootPtr->left, a, b);
        range_count(rootPtr->right, a, b);
    }
}

```

5. (10%) Write a function `int range_count (struct binarytree* rootPtr, int a, int b)` which accepts a pointer `rootPtr` to the root of a binary search tree `T` and returns the number of nodes in `T` having key values between `a` and `b` ($a \leq \text{key} \leq b$). The node structure of the binary search tree is defined as

```

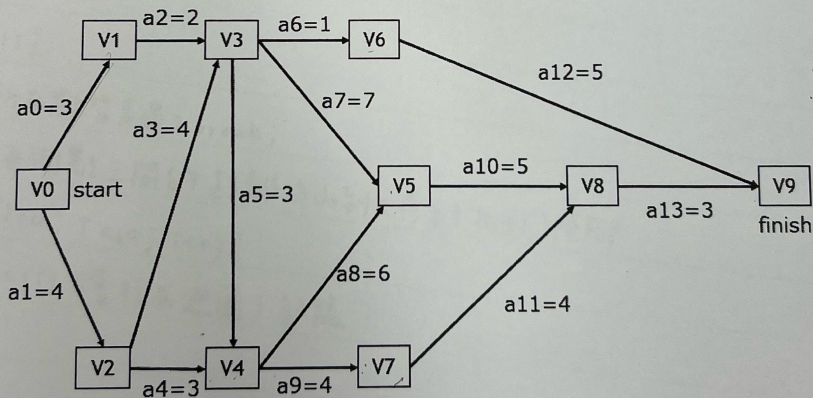
struct binarytree {
    int key;
    binarytree *left, *right;
};

```

You may call other user-defined functions in the function `range_count`.

6. (6%) Please design a program to add a new node 't' to the right of node 's' in a threaded binary tree.

- X → 不考，沒教 (15%) For the AOE (activity on edge) network shown below, do the following
- Compute the early starting time for each activity.
 - Compute the late starting time for each activity.
 - Determine the critical path of the project.



(a)

$V_1 = 3$
 $V_2 = 4$
 $V_3 = 5$
 $V_4 = 7$
 $V_5 = 12$
 $V_6 = 6$
 $V_7 = 11$

$V_8 = 15$
 $V_9 = 11$

(b)

$V_1 = 3$
 $V_2 = 4$
 $V_3 = 7$
 $V_4 = 10$
 $V_5 = 16$
 $V_6 = 8$
 $V_7 = 14$

$V_8 = 21$
 $V_9 = 24$

1、

卡塔蘭數 $C_n = \frac{(2n)!}{(n+1)!n!}$

(1) different btree: $n!C_n$

$$3! \times \frac{\cancel{6} \times 5}{\cancel{3} \times \cancel{2} \times 1} = 6 \times 5 = 30$$

(2) different ordered tree: $n!C_{n-1}$

$$3! \times \frac{4!}{3!2!} = 6 \times 2 = 12$$

(3) different oriented tree: n^{n-1}

$$3^2 = 9$$

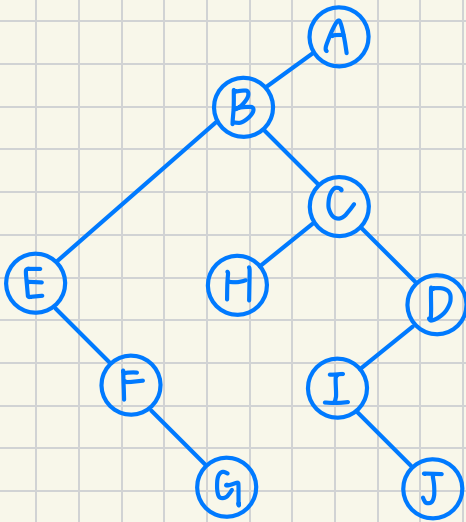
(4) different free tree: n^{n-2}

$$3^1 = 3$$

(5) 3 traversal way, different btree? C_n

$$\frac{6!}{4!3!} = \frac{\cancel{6} \times 5}{\cancel{3} \times \cancel{2}} = 5$$

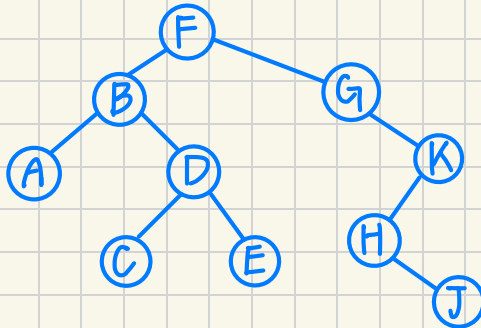
2,
(a)



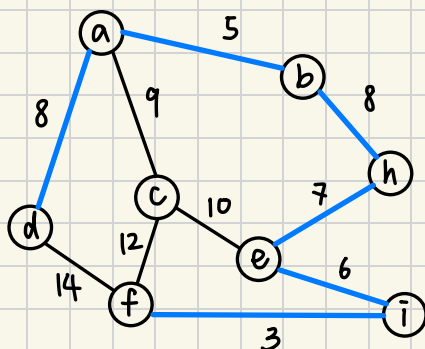
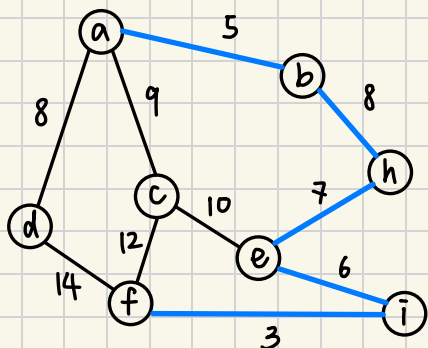
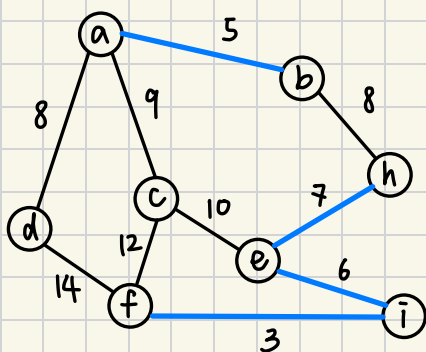
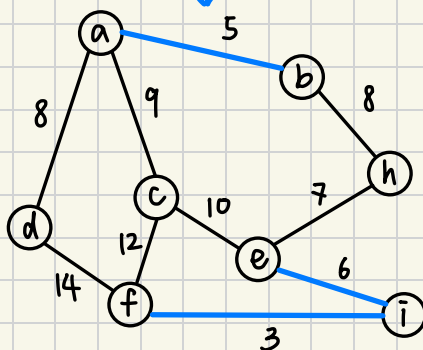
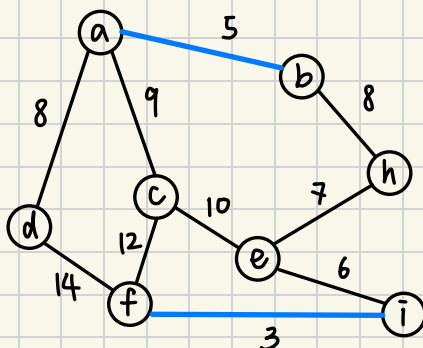
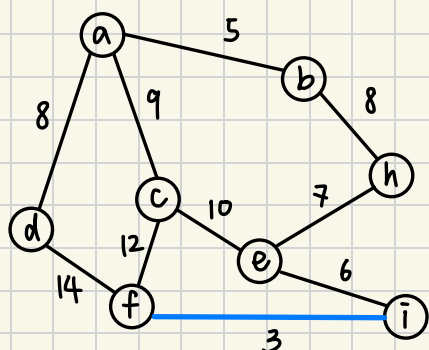
(b) postorder LRV!

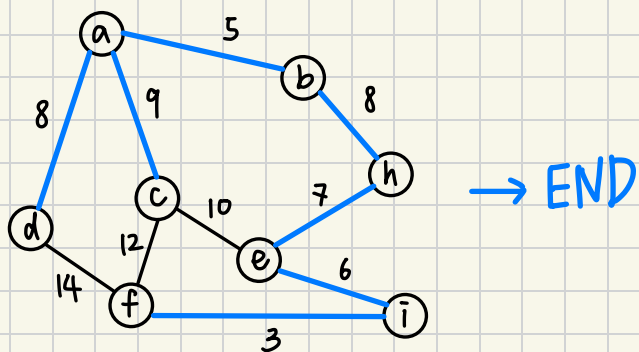
$AB / C * DE^{-+}$

(c)
 LVR inorder : $ABCD E \underline{F} GHJK$
 LRV post-order : $ACEDBJH K G \underline{F}$

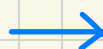
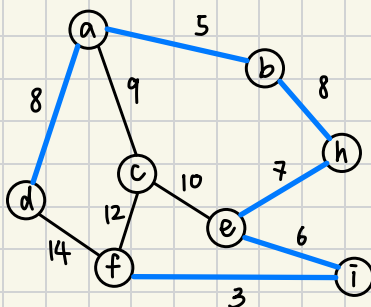
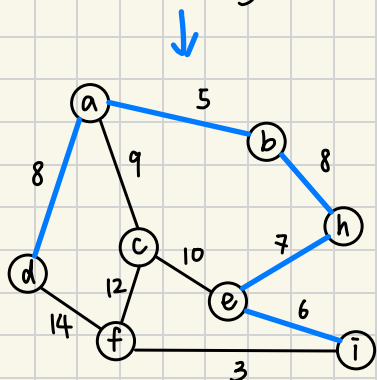
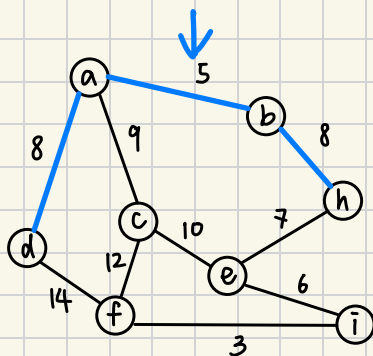
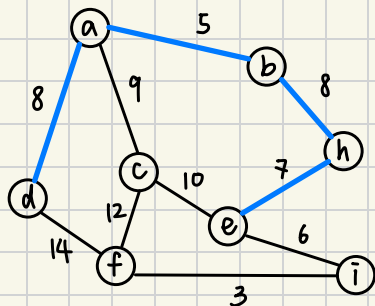
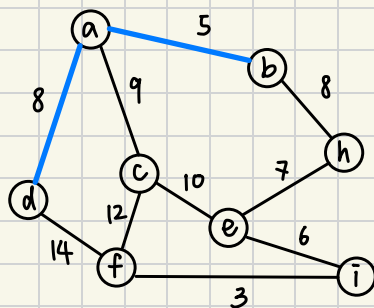
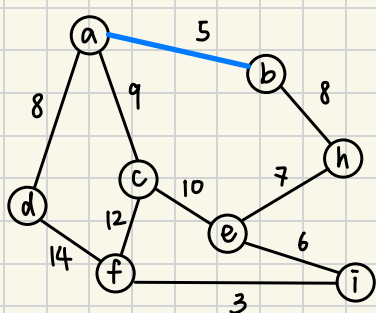


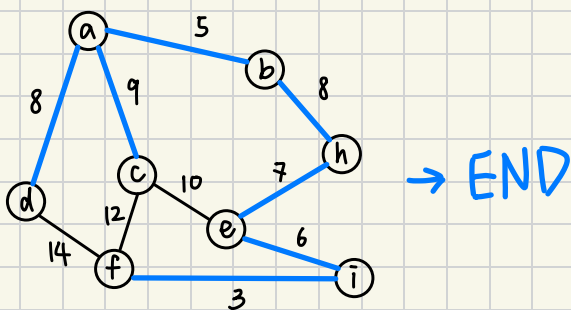
3. ~~2~~ ~~5~~ ~~8~~ ~~7~~ ~~8~~ ~~2~~ 9 10 12 14
 (a) kruskal's → 從最小w邊開始找



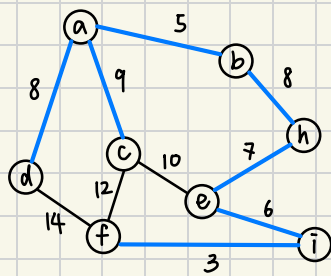
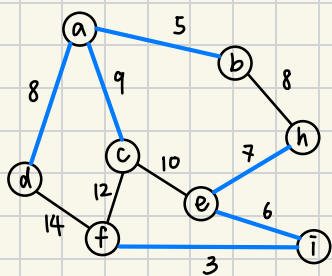


Prim's → 每次從新分量中找最小的邊





sollin → 將所有 nodes 視成獨立連通分量
每次為連通分量選最小 w 的邊



→ END

(b)

Algo_kruskal:

$MST = \{ \}$

$E = \{ \dots \}$ // 邊集合

sort(E , 依權重由小到大)

foreach (u, v) in E :

if (加入邊 (u, v) 不會使 MST 有環)

$MST.add(u, v)$

4

```
void dinsert ( node_pointer node, node_pointer newnode) {
```

```
(1) newnode → llink = node → llink
```

```
(2) newnode → rlink = node
```

```
(3) node → rlink = newnode
```

```
(4) newnode → llink → rlink = newnode
```

```
}
```

```
void ddelete (node_pointer node, node_pointer deleted) {
```

```
if (node == deleted)
```

```
    printf("Deletion of head node not permitted\n");
```

```
else {
```

```
(5) deleted → llink → rlink = deleted → rlink ;
```

```
(6) deleted → rlink → link = deleted → llink ;
```

```
free(deleted);
```

```
}
```

```
}
```

5.

```
int range-count(struct binarytree* rootPtr, int a, int b){  
    if (rootPtr == nullptr) return 0;  
  
    if (rootPtr->key < a){  
        return range-count(rootPtr->right, a, b);  
    }  
  
    if (rootPtr->key > b){  
        return range-count(rootPtr->left, a, b);  
    }  
  
    return 1 + range-count(rootPtr->left, a, b) +  
           range-count(rootPtr->right, a, b);  
}
```

6、

```
void insertRight (tbNode *s, tbNode *t){  
    t→rightC = s→rightC; // upd 孩子  
    t→rightT = s→rightT; // upd 引線  
    t→leftC = s; // 设新引線  
    t→leftT = true;
```

```
    s→rightC = t;
```

```
    s→rightT = false;
```

```
    if (!t→rightT){ // 若t後还有子樹,修正引線
```

```
        tbPtr tp = t→rightC;
```

```
        while (!tp→leftC) tp = tp→leftC;
```

```
        tp→leftC = t;
```

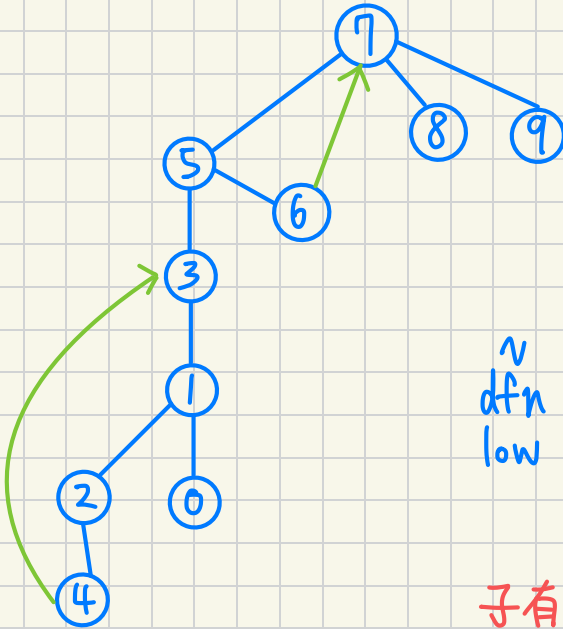
```
    }
```

```
}
```

HW 4 (Extend)

1.

DFS: $7 \rightarrow 5 \rightarrow 3 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 9$



						$\text{low}(6)=0$				
					$\text{dfn}(3)$	$\text{dfn}(7)$				
				$\uparrow$	$\uparrow$	$\uparrow$				
$\sim$	0	1	2	3	4	5	6	7	8	9
dfn	4	3	5	2	6	1	7	0	8	9
low	4	2	2	2	2	0	0	0	8	9
				$\downarrow$						

$\min(\text{dfn}(2), \text{low}(4))$

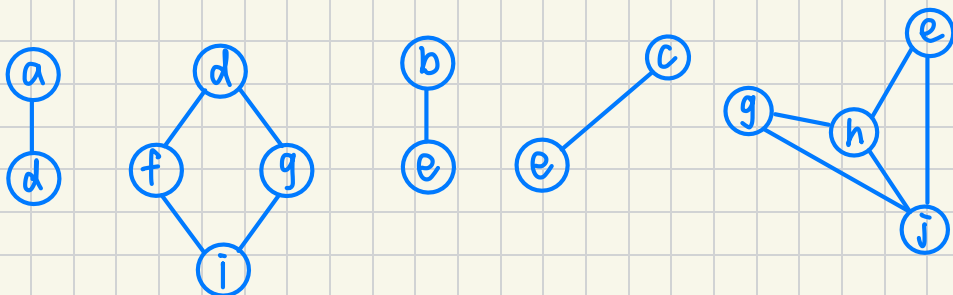
子有回边、low更小, 父直接拿子的low

dfn: dfs 到访 顺序

2,

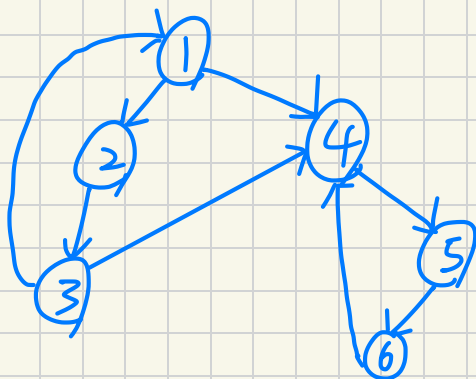
(a) d, g, e

(b)



3,

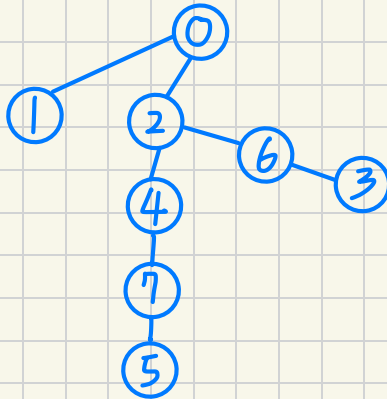
	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	1	0	0	0
3	1	0	0	1	0	0
4	0	0	0	0	1	0
5	0	0	0	0	0	1
6	0	0	0	1	0	0



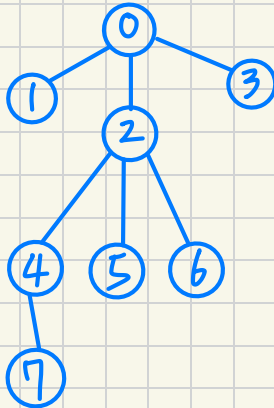
4.

(a)

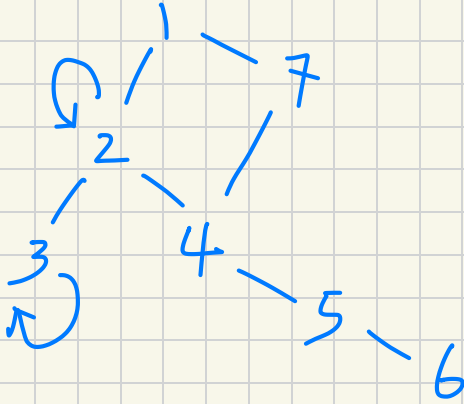
DFS: $0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 6 \rightarrow 3$



(b) BFS: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$



5

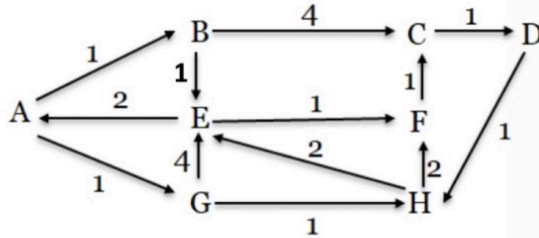


(a) BFS: $v_1 \rightarrow v_2 \rightarrow v_7 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5 \rightarrow v_6$

(b) DFS: $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5 \rightarrow v_6 \rightarrow v_7$

6、Dijkstra 前提是没有负边

6. (10%) A weighted directed graph G is given as follows. Filling in the given table using Dijkstra's algorithm to find shortest path from A to all nodes. If more than 1 vertices have the same minimum distance, choose the vertex with alphabet priority.

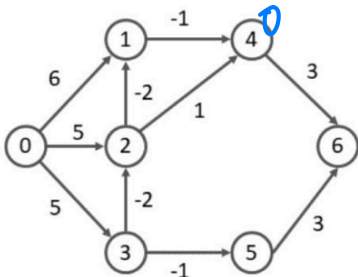


Iteration	Vertex Selected	Distance							
		A	B	C	D	E	F	G	H
Initial	A	0	1	∞	∞	∞	∞	1	∞
1	B	0	1	5	∞	2	∞	1	∞
2	G	0	1	5	∞	2	∞	1	2
3	E	0	1	5	∞	2	3	1	2
4	H	0	1	5	∞	2	3	1	2
5	F	0	1	4	∞	2	3	1	2
6	C	0	1	4	5	2	3	1	2
7	D	0	1	4	5	2	3	1	2

每次都未選过的人中挑最小的去鬆弛別人

8.

8. (15%) A weighted directed graph G is given as follows. Filling in the given table using Bellman-Ford algorithm to find shortest path from 0 to all nodes.



用數

k	$dist^k[u]$						
	0	1	2	3	4	5	6
1	0	6	5	5	∞	∞	∞
2	0	3	3	5	5	4	∞
3	0	1	3	5	2	4	7
4	0	1	3	5	0	4	5
5	0	1	3	5	0	4	3
6	0	1	3	5	0	4	3

這裡算6時要用上一輪的

Bellman ford
先建出用表

0: 1(6), 2(5), 3(5)

1: 4(-1)
2: 1(-2), 4(1)
3: 2(-2), 5(-1)

4: 6(3)

5: 6(3)

6: X

<key> 只關注上一輪
有更新的點，後relax